Apartment 50: Paper 005

AuthorityEffect: None. This is fictional examination staging, not a historical quotation or a source of mathematical authority.

Landau points at the declared carrier. Woland asks which claim its checker can establish. Behemoth asks for the smallest valid near miss. Their task in this companion is **a bounded checked-proposal session**.

Begin with the manuscript's worked example, then attempt the eight-level quiz before reading the study guide and instructor key. Finish with its executable capstone. Arrows sketch graph edges; they do not automatically attach higher-dimensional cells. A receipt verifies its stated finite check and grants no authority to extend Genesis or make a physical claim.

The examiner's standard is a concrete calculation, an explicit counterexample, or a source-bound missing obligation. The manuscript source reading governs the answer.

Study guide and instructor key: Paper 005

AuthorityEffect: None. Read the fictional Apartment 50 preface and attempt the quiz before using the key.

Learning route and notation

Read manuscript Sections 1-2 and solve Levels 0-1 on paper. Read Sections 4-5 and attempt Levels 2-3 against the move table. Read Sections 3, 6, and 9 before Levels 4-6. Finish Level 7 by running the capstone and auditing its actual output. The examiner's fictional role supplies no mathematical authority.

Throughout, the ordered directed edges are (ab,bc,ca), coefficients are rational, c=(1,1,1), and w=(1,2,3), unless a question explicitly changes them. No two-dimensional cells are declared. Let f=(f_a,f_b,f_c), df=(f_b-f_a,f_c-f_b,f_a-f_c), r=c-df, and D_c(f)=sum_e w_e |r_e|. The quotient minimum Phi_w(c) minimizes over potentials while keeping c fixed. A finite graph does not make the set of rational potentials finite.

Object or move	What can change	What it checks or preserves
Potential or section CUT	Vertex potential	Fixed claims and their cycle sums; accepts non-increase of the current discrepancy
Replacement-claim CUT	All edge claims	Accepts non-increase of the quotient minimum; needs an explicit complete exact map
ZERO_OBSTRUCTION	No active data	Whether the stored claims admit some exact potential
ZERO_DEFECT	No active data	Whether the currently stored potential fits the stored claims
EDGE_MATCH:u,v	No active data	One declared directed edge
RETRACT	Active section and claims	Restores a real admitted snapshot; can increase the score

Level 0 key: Declare the instance

1. V=(a,b,c), E=(a to b,b to c,c to a), ordinary rational addition, c=(1,1,1), w=(1,2,3), a common offset unit, and no attached faces. State whether potential

adjustment, replacement of claims, or both are permitted. A modulo-clock convention changes the coefficient interpretation and equality relation. The rational telescoping calculation cannot be reused unchanged for a modular model.

2. Weights price discrepancy; they do not establish truth. Before changing a measured claim, specify whether it is a revisable proposal, an uncertain observation with a declared correction model, or a frozen observation. An allowed code operation is not itself a scientific justification for replacing an observation.

3. D_c(0)=1+2+3=6. At f=(0,-2,-1), df=(-2,1,1) and r=(3,0,0), so the score is three. Question 5 supplies the matching lower bound, hence Phi_w(c)=3. Six is the current all-zero-potential score; three is the best score for the same claims.

4. df=(-2,1,1), r=(3,0,0). Selecting f leaves sum(c)=3 and Phi_w(c)=3. Replacing c by c'=df gives sum(c')=0 and Phi_w(c')=0. With the displayed f, D_c'(f)=0; with the zero potential, D_c'(0)=2+2+3=7. A zero minimum and a nonzero current discrepancy can coexist.

Level 1 key: Certify the small calculation

5. For every potential, sum(df)=0, hence sum(r)=3. Positive weights with minimum one give

$$D_c(f) = \sum_e w_e|r_e| \geq \sum_e |r_e| \geq \left|\sum_e r_e\right| = 3.$$

The witness f=(0,-2,-1) attains three. The lower bound ranges over every potential; one matching feasible witness proves optimality. Testing several potentials supplies an upper bound only unless an independent argument proves the candidate family exhaustive.

6. q dot df=(f_b-f_a)+(f_c-f_b)+(f_a-f_c)=0. Since |q_e|=1<=w_e, for every f,

$$q \cdot c = q \cdot (c - df) \leq \sum_e |q_e|\,|r_e| \leq D_c(f).$$

Here q dot c=3. This is a concrete feasible dual lower-bound witness, not an assertion of a new duality theorem. The potential in Question 5 supplies the upper bound three, closing the gap. Negating q would give the useful bound when the cycle sum is negative.

7. Adding (t,t,t) changes the displayed vertex values but no differences, residuals, or scores. In this weighted triangle, equality in the bound forces the residual onto the sole cheapest edge, giving (3,0,0). The absolute potential is still free up to an additive constant. The cheapest edge determines the minimizing residual here, not the truth of a claim or a physically preferred clock origin.

8. Replace ca=+1 by ac=-1. Its potential difference becomes f_c-f_a, the negative of the old f_a-f_c. The oriented cycle constraint is c_ab+c_bc-c_ac=3. Retaining ac=+1 instead produces a different instance with cycle mismatch one. Reversing an arrow without its value changes the data rather than relabeling it.

Level 2 key: Admit and refuse proposals

9. The original minimum is three. The bad replacement has sum five, minimum five, and returns `DefectIncrease`, admitted=False. The replacement (-2,1,1) has minimum zero and returns `ExactRepair`, admitted=True, with 3 to 0. ZERO_OBSTRUCTION verifies stored replacement claims. RETRACT restores the original minimum three. Save the actual `step_id` returned by the final admitted ASSERT; counters can advance on refusals and are not a substitute for an actual snapshot identifier.

10. Both incomplete maps return `IncompleteProposal`. Active claims, section, and admitted history remain unchanged. Initialization explicitly scores unasserted declared edges as zero; a replacement CUT has a different completeness contract and must name every declared edge. The omitted ca cannot be interpreted as a newly supplied zero.

11. The new cycle sum is two. The same bound gives Phi_w=2, attained by putting residual two on ab. A witness is f=(0,-2,-1), whose differences are (-2,1,1), leaving residual (2,0,0) against (0,1,1). From the original minimum three, the complete replacement is admitted as `MonotonicDescent`, 3 to 2. Its obstruction is still positive; it is not `ExactRepair`.

12. The cycle sum is still three, so the minimum is three. For example, f=(0,-1,0) has differences (-1,1,0) and residual (3,0,0). This replacement is admitted as `IdempotentStationary` and changes the claims. The status records equality of the checked score, not equality of all state or strict search progress.

Level 3 key: Identify the checked obligation

13. The section CUT lowers D_c from six to three and is admitted as `MonotonicDescent`. Phi_w(c) remains three, because the claims are unchanged. Reading every verdict's `defect_before` as a quotient minimum would therefore be wrong: section CUT verdicts use the section discrepancy; relational CUT verdicts use the quotient minimum.

14. For stored c'=(-2,1,1) and section zero, Phi_w(c')=0 but D_c'(0)=7. ZERO_OBSTRUCTION passes and ZERO_DEFECT fails. For the original claims and minimizing f=(0,-2,-1), the bc difference is one and EDGE_MATCH:b,c passes; the global minimum remains three, so ZERO_OBSTRUCTION fails. One correct edge does not discharge the cycle inconsistency.

15. All three obligations return `VerificationFailed`: d,e exceeds the carrier,

a,c is the undeclared reverse of ca, and a is malformed. The patched verifier checks the declared directed edge before comparing values. Looking up absent endpoints with default zeros could otherwise produce 0=0 and a false verification.

16. Both payloads return `MoveViolation`; the undeclared section key returns `CarrierViolation`; the Boolean value returns `IncompleteProposal`. Active state remains unchanged. Section CUTs retain the legacy convention that omitted declared vertices have zero potential, so omission of a section vertex is not the replacement-claim completeness violation of Question 10. Supply full potentials in these exercises to avoid hidden defaults.

Level 4 key: Respect the execution route

17. The hand argument proves the triangle's minimum from a universal bound and a feasible potential. The printed worksheet runs exact arithmetic. The governor's offline route computes a result by Python enumeration and records `python-enumeration` in the demo output. None is a fresh native functional execution or a source-to-binary attestation. The native-only quickstart requires explicit pins and cannot silently use the offline route.

18. A configured native refusal or error propagates; Python cannot substitute an accepted result for that step. If projection fails after candidate scoring, the patched replacement CUT restores the old active claims before propagating the error, leaving the section and admitted history unchanged. A mocked adapter regression tests this failure boundary; it does not claim to run a native executable. An attempt counter may still advance.

19. Supported Edge object keys normalize to (u,v) before either scoring route. The three unit claims still have minimum three, rather than disappearing into an all-zero lookup. An added undeclared (d,e) claim raises `ValueError` before scoring; silently dropping it would change the proposed instance.

20. `compute_phi1` rejects duplicate vertex names and duplicate directed edge pairs with `ValueError` before choosing a calculator. This is its scoring boundary, not a claim that every legacy constructor validates the whole instance. The reverse pair (b,a) differs from (a,b) and is permissible if separately declared with its own oriented claim. No simplex face or antisymmetry relation is inferred automatically.

Level 5 key: Bound the inference

21. The streaming harness accepts integer 1 and exact rational string `"1/10"`, producing exact rationals. It refuses JSON float 0.1 and Boolean true before scoring. Fraction(0.1) represents the float's binary approximation, not the intended one tenth. Reconstructing an exact fraction from an already rounded observation does not recover the missing input intention. At the governor API, use an integer or Fraction rather than a string.

22. Under the separately declared exercise policy, return UNKNOWN with the attempt bound, exact attempts and refusals, surviving or unexamined candidate family, and remaining obligation. Do not claim impossibility or make an unbudgeted third attempt. This is an instructional search policy; the four-move governor and demo have neither an UNKNOWN terminal status nor a proposal budget controller. A mathematical obstruction would instead require a proof such as Question 5 for a precisely fixed instance and permitted operation.

23. Non-increase permits stationary repetition. Strict rational descent also need not terminate: 1,1/2,1/4,… is infinite and positive. A finite state space with strict descent, a well-founded discrete measure, a positive minimum score decrement together with a lower bound, or an explicit finite attempt budget can justify an appropriate termination statement. A budget can force a stop with unresolved work; it does not prove a solution.

24. With equal weights, f=(0,-2,-1) yields residual (3,0,0); f=(0,1,-1) yields differences (1,-2,1), hence residual (0,3,0). Both have cost three and attain the same lower bound. There are further tied residuals, including mixtures with nonnegative entries summing to three. An implementation may choose a representative by traversal or tie policy; optimal cost does not select a factually true statement.

Level 6 key: Keep evidence in custody

25. The pin binds the executable bytes read by the adapter to the caller's stated SHA-256. It does not prove those bytes arose from the intended source, identify an unsigned producer, or make their answers true. Record source revision, dependency lock, build toolchain, command, and output. A reproducible-build comparison or an independently authenticated attestation would address different additional obligations.

26. Attribute the saved run to its earlier source, executable, and test scope. It is useful historical evidence, not a fresh pass for the patched package. Run the current suite and record pass, fail, and skip counts separately. Missing native configuration is unavailable execution evidence; a skip is not a native pass. The current publication review has no native execution.

27. The verifier establishes the stated file census, recorded byte hashes, and ZIP membership/envelope checks. It does not run all mathematical tests by default, validate a domain encoding, prove every answer, or measure a live model. Full JCS conformance also requires RFC 8785's complete serialization contract; sorted keys alone are insufficient.

28. Record the following fields with their actual source:

Field	Required observation
Instance	Ordered vertices and edges; exact c and w; coefficient and unit conventions; allowed changes
Inputs and state	Each actual proposal, exact refusal or admission, relevant score, active state, saved snapshot ID
Calculator	Selected path; explicit native executable digest if a native route executes
Bindings	Source revision plus hashes of the tested package files; dependency and tool versions
Execution	Exact command and working directory, exit status, stdout and stderr, run time
Scope	Tests passed, failed, or skipped and why; native and provider execution availability

Read snapshot IDs, refusals, commands, exit status, outputs, and executable digests from the actual run. Do not fabricate them from the expected table. The governor's admitted history omits refusals and VERIFY records, so preserving that history alone is insufficient custody for the complete session.

Level 7 key: Transfer and audit

29. Treat vertices as events, use a common time unit, declare edge directions and rational additive offsets, and keep measured c frozen. Changing f may minimize the residual, but it cannot change the cycle sum three or make the minimum zero. Revising observations needs a separately justified uncertainty, calibration, or data-correction model with its own evidence. A zero score after replacing measurements answers a different question.

30. Deleting an edge changes the carrier and can destroy the cycle; zeroing a weight violates the positive-weight instance. Neither is a repair of the original declaration. Reject the proposal under the existing contract, or explicitly declare, price, and analyze a different instance. A lower score caused by deleting the obligation does not discharge it.

31. A native search-bound refusal establishes that this evaluator did not complete its declared exhaustive check. It is not an impossibility proof or an accepted partial minimum. This configured step cannot switch to Python. Starting a separately declared offline route would require its own scope and record. The offline enumerator is exact when it completes, but has no built-in proposal

budget or equivalent native forest cap and may grow combinatorially.

32. Run the capstone script from the package directory:

```
python code/study_capstone.py
python -m pytest code/tests/test_manual_contracts.py -q
```

Use `py -3.12` on Windows if needed. The script uses the same calculator-selection rule as play_demo; configured native errors propagate. Without native configuration it prints `python-enumeration`. Preserve its actual snapshot identifier and output. The expected transitions are:

Action	Verdict or observation	Score or active state
Three ASSERTs	Asserted	Original minimum 3; snapshot saved from the final ASSERT
Empty replacement CUT	IncompleteProposal, refused	Original claims preserved
Replacement (5,0,0)	DefectIncrease, refused	Proposed minimum 5; original claims preserved
Replacement (0,1,1)	MonotonicDescent, admitted	3 to 2
Replacement (-2,1,1)	ExactRepair, admitted	2 to 0
ZERO_OBSTRUCTION	Verified	Stored consistent replacement
RETRACT actual snapshot	Retracted	Original claims (1,1,1), minimum 3

The strongest result is that this declared finite session discriminates the stated refusals, accepts the permitted replacements, and restores the saved instance on the recorded calculator path. It does not establish the external truth of the replacement, universal search success, or a measured model improvement.

Verification and grading boundary

Use the quiz's local two-point rubric, seven of eight points per level, and scope gates. The publication review and its JSON record separate current execution from the earlier receipts. The deposit verifier checks file custody; run the named tests for implementation evidence. `test_manual_contracts.py` exercises the capstone, wrong-carrier refusals, exact inputs, verification scope, stationary replacement, and projection-error rollback. A successful finite replay is neither a general theorem, a provider trial, a learner study, nor an authorization grant.

Theoretical minimum: Paper 005

AuthorityEffect: None. Read the fictional Apartment 50 preface. The source reading supplies the definitions; the study guide contains a separate instructor key.

Use the ordered directed edges (ab,bc,ca), rational claims c=(1,1,1), positive weights w=(1,2,3), and ordinary additive potentials unless a question explicitly changes them. The symbol `a>b>c>a` sketches edges; it declares no filled face. Give exact calculations and identify the move or obligation being checked. All implementation questions refer to this package's patched governor, not every repository surface.

The examination has eight levels, four questions each. Attempt it without the instructor key. The budget in Question 22 is a separately declared exercise policy; UNKNOWN is not a terminal status implemented by this four-move governor.

Level 0: Declare the instance

1. List the vertices, ordered edges, coefficient system, claims, weights, units, allowed changes, and cell declarations needed to reproduce this example. Which datum would an undisclosed wraparound-clock convention change?
2. Is a weight evidence that one claim is true? What additional modeling decision is needed before replacing a measured edge claim?
3. Compute the score D_c(0) and the minimum Phi_w(c). Explain why six and three can both be correct reports about the same unchanged claims.
4. Compute df and r for f=(0,-2,-1). Contrast selecting this potential with replacing claims by c'=(-2,1,1). State each resulting cycle sum and minimum.

Level 1: Certify the small calculation

5. Prove the lower bound three for every potential, and give a witness attaining it. Distinguish that proof from testing a few potentials.
6. Let q=(1,1,1). Show q dot df=0 and |q_e|<=w_e, then use q dot c to obtain a lower bound on the weighted discrepancy. What completes the optimality argument?
7. Replace f by f+(t,t,t), with t rational. What changes? Does the unique cheapest edge make the absolute potential unique?
8. Reverse ca into ac. Give the correctly transformed third claim and potential difference. What false conclusion could result from keeping the claim +1 after reversing its arrow?

Level 2: Admit and refuse proposals

9. Trace ASSERT (1,1,1), replacement CUT (5,0,0), replacement CUT (-2,1,1), VERIFY ZERO_OBSTRUCTION, and RETRACT to the saved post-ASSERT snapshot. Give the minima and admission outcomes; explain where the snapshot identifier comes from.
10. After ASSERT, propose either an empty replacement map or just ab=-2, bc=1. Give the refusal and active-state outcome. Why does the initialization default for unasserted edges not authorize this CUT?
11. Propose the complete replacement (0,1,1). Compute its minimum and the verdict from the original instance. Is it an exact repair?
12. Propose the complete replacement (2,1,0). Compute its minimum and verdict. Does `IdempotentStationary` establish that the claims were unchanged or the proposer has made progress?

Level 3: Identify the checked obligation

13. Use a section CUT to move the original all-zero potential to (0,-2,-1), with claims fixed. Which score decreases, and which minimum remains unchanged?
14. Install the consistent claims (-2,1,1) while the stored section is zero. Compare ZERO_OBSTRUCTION and ZERO_DEFECT. Then explain why EDGE_MATCH:b,c can pass at the original minimizing potential while the original ZERO_OBSTRUCTION fails.
15. Try EDGE_MATCH:d,e, EDGE_MATCH:a,c, and the malformed EDGE_MATCH:a. What should the patched verifier return, and why is default-zero comparison insufficient?
16. A CUT supplies both a section and replacement claims. Another supplies an undeclared section key d, and a third supplies a Boolean section value. Name the respective refusals and preserved active state.

Level 4: Respect the execution route

17. The native calculator is unavailable. What can the hand argument, the printed worksheet, and the Python enumeration session each establish? Which execution claim remains unavailable?
18. A native executable is configured but refuses during projection after a replacement's score was calculated. May Python substitute a result? What must happen to the pre-CUT active claims, section, and admitted history?
19. Represent the three unit claims with supported `Edge` object keys rather than tuple keys. What minimum should result after normalization? What must happen to an additional undeclared claim (d,e)=9?
20. Does scoring a graph with a duplicate vertex or duplicate directed edge produce a defensible minimum? Identify the check boundary in this package. Distinguish duplicate directed edges from the separately declared reverse edge (b,a).

Level 5: Bound the inference

21. Compare the harness inputs `"1/10"`, integer 1, JSON number 0.1, and JSON true. Which are accepted as exact claim values? Why is converting a float to `Fraction` insufficient?
22. Under a separate budget of two proposal attempts, both fail and candidates remain unexamined. Give the scoped terminal report. Can two refusals certify global impossibility, or authorize a third unbudgeted attempt? Is this policy already implemented by the demo?
23. Does accepting non-increase prove finite termination? Would strict descent through 1, 1/2, 1/4, … prove it over the rationals? State an additional condition that would support termination.
24. Change the weights to (1,1,1). Give two different minimizing residuals for the original claims and potentials attaining them. Does equal optimal cost select the true event statement?

Level 6: Keep evidence in custody

25. What does an executable SHA-256 pin establish? What source-to-binary, producer-authentication, and semantic obligations remain?
26. A saved record reports 75 passing tests against an earlier Windows executable. The current package has new patches. What can be claimed now? Explain why a skipped native test cannot be counted as a passing native run.
27. Both manifests and the deterministic ZIP verify. Does that validate the graph interpretation, all quiz answers, a live model comparison, or full RFC 8785 conformance? State the narrower finding.
28. Design a minimal reproduction record for this session. Include calculator selection, exact instance, source and file bindings, commands and tools, refusals, admitted state, snapshot restoration, and test skips. Which values must come from execution rather than reconstruction?

Level 7: Transfer and audit

29. Translate the example into measured time offsets that are frozen observations. Which permitted potential operation remains meaningful? Can it make the original minimum zero? What extra model would be needed to justify changing observations?
30. A proposer deletes the cheapest edge or sets its weight to zero to obtain a better score. Is that a repair of the declared instance? Describe the required response or explicit new declaration.
31. The native spanning-forest search reaches its bound and refuses. What inference is justified, and may the same configured step switch to Python? Is the offline enumeration route a bounded search controller?
32. Execute the capstone: ASSERT original claims; save its actual snapshot; refuse an empty CUT; refuse (5,0,0); admit (0,1,1); admit (-2,1,1); VER-

IFY; RETRACT. Record the path, verdicts, score transitions, and restored claims. State the strongest supported result and one claim it does not establish.

Local grading rubric

Award two points per question: one for the exact calculation or correct implementation outcome, and one for the declared scope or missing obligation. A level passes at seven of eight points with no unsupported acceptance. This threshold is an instructional choice, not an empirically validated measure of reasoning ability.

Do not pass the corresponding level if an answer silently changes the carrier, treats omitted replacement claims as zeros, replaces a configured native refusal with an accepted Python value, promotes an unfinished search to impossibility, or treats a checksum as semantic proof. Mark an unsupported step as an open obligation; return the implementation's actual refusal for a violated input contract. Keep mathematical obstruction, runtime refusal, and unavailable execution evidence distinct.